Git và GitHub

1. Git là gì? Khác GitHub chỗ nào?

Git

- Là một hệ thống quản lý phiên bản phân tán.
- Một mô hình hệ thống, các máy tính có thể clone lại mã nguồn từ một repo

Git Hub

- Là một dịch vụ lưu trữ trên web dành cho các dự án có sử dụng kiểm soát Git revison
- Một công ty cung cấp dịch vụ máy chủ repo công cộng, mỗi ~ có thể truy cập vào website trang chủ, tạo tk và tạo ra kho chứa source code của riêng mình

2. Các trạng thái file trong Git.

- Committed: Dữ liệu đc lưu trữ 1 cách an toàn (.) cơ sở dữ liệu (Commit thành công)
- Staged: Là bạn đã đánh dấu sẽ commit phiên bản hiện tại của 1 tập tin đã chỉnh sửa trong lần commit sắp tới. Trạng thái này xảy ra khi sử dụng lệnh git add - nhưng chưa commit.
- Modified: Đã thay đổi tập tin nhưng chưa commit vào cơ sở dữ liệu, tức là bạn chưa sd lệnh git add và git commit.

3. Các câu lệnh.

- Git init: Khởi tạo một kho chứa (Repository) Git mới trong thư mục hiện tại từ tạo ra 1 thư mục .git (nơi git lưu trữ toàn bộ về lịch sử commit, cấu hình, các nhánh)
- Git add: Là lệnh dùng để đưa các thay đổi của file vào staging area (khu vực chuẩn bị) trước khi commit
  - Chọn file cần theo dõi: Git chỉ quản lý những file bạn thêm vào staging area
  - Thêm 1 file ưu thế: git add filename.txt
  - Thêm nhiều file: git add filename1.txt filename2.txt
  - Thêm toàn bộ thay đổi trong thư mục hiện tại: git add .
    ⇒ Stage tất cả file mới, sửa đổi, hoặc xoá trong thư mục hiện tại.
  - Thêm toàn bộ thay đổi trong repo: git add -A
    ⇒ Stage tất cả các thay đổi (.) toàn bộ repo, bao gồm cả việc xoá file
  - Thêm file (.) 1 thư mục ưu thế: git add src/
    ⇒ Stage toàn bộ file (.) thư mục: src → ó thể đặt tên khác

- Git commit: Có nghĩa lưu lại những thay đổi bạn đã ~~lưu~~ chuẩn bị (staged) vào lịch sử của repository.

• Commit với message: git commit -m "message"

• Commit tất cả thay đổi: Git commit -am "message"

Có thể bỏ qua bước git add

~~Nhưng~~ lưu ý:

Giả sử bạn có 2 file main.py và utils.py và đã sửa chúng

- Thay vì chạy: git add main.py utils.py

git commit -m "message"

Có thể rút gọn: git commit -am "message"

Nhưng nếu tạo thêm file mới: VD test.py, thì dòng lệnh trên sẽ không commit file mới đó. Bạn lúc này cần đến: git add test.py

git commit -m "..."

• Commit nhiều dòng thông điệp git commit

⇒ Thay vì commit message nhiều dòng, thay vì dùng -m" " có thể dùng git commit

Khi gõ xong sẽ xuất hiện 1 trình soạn thảo giống vim (sau sẽ biết)

- Tìm dòng đầu tiên không có dấu # nhập message và ấn esc nhập

:wq để lưu và thoát

:q! để không lưu và thoát

• Sửa lại commit gần nhất: git commit --amend

vd:

git commit -m "Tung Dz" // nhấn message

git commit --amend // để sửa sau đó hiển thị bằng vim

Tương tác sử dụng như của git commit trên

vd: Thêm file bị sót

Giả sử bạn commit một file nhưng quên ~~commit~~ add 1 file ≠ vào

git file1.txt

git commit -m "thêm file 1"

Sau đó bạn nhớ ra còn file 2, gõ:

git ~~commit~~ add file2

git commit --amend

Mở ra bằng vim nhập lại message

⇒ Nói ngắn gọn: git commit --amend để chỉnh sửa commit gần nhất đồng thời có thể bổ sung thêm file nữa

Scanned with CamScanner

• Commit tất cả thay đổi đã staged : git commit -a

=> Cú pháp khác commit -am nhưng bản chất là 1

- Git config: Dùng để thiết lập thông và cấu hình cho git (ví dụ tên, email, editor mặc định)

Ý nghĩa của nó: Lưu thông tin tác giả commit.

• tên: git config --global user.name "Trần Thanh Tùng"

• email: git config --global user.email "thanhtung@gmail.com"

Đặt môi trg làm việc:

• chọn ~~môi trường~~ editor làm việc: git config --global core.editor "vim"

• Quản lý màu sắc, alias, ... (Thg k° cần)

phạm vi cấu hình:

--global: Toàn bộ máy (toàn bộ repo của 1 user trên máy đó)

--local: áp dụng cho repo hiện tại (repo cụ thể)

--system: Áp dụng cho toàn bộ hệ thống (mọi user và mọi repo trên cùng 1 máy)

Lưu ý nếu k° config tên và email, Git sẽ không cho ~~config~~ commit

- Git ~~clo~~ clone: Sao chép toàn bộ một repo từ máy chủ (vd GitHub, GitLab, ...) về máy tính của bạn. Nó sẽ tạo ra 1 bản sao đầy đủ

Toàn bộ mã nguồn
Lịch sử commit
Các nhánh
Cấu hình git liên quan đến repo đó

Cú pháp cơ bản: vd vào Github trên website, copy URL

git clone <URL>

vd git clone https://github.com/.....

- Git ~~stage~~ status: Hiển thị trạng thái của repository

• cho biết file nào đã được chỉnh sửa nhưng chưa stage

• cho biết file nào đã được stage.

• Báo cáo các file mới chưa được Git theo dõi

• Hiển thị nhánh đang làm việc.

Scanned with CamScanner

- Git push: Upload những file được commit trong lịch sử của repo lên sever (GitHub)

Cú pháp: git push <remote> <branch>

remote: Một bản sao của Repo cục bộ nhưng trên sever

branch: Tên nhánh

Thiết lập remote lần đầu (nếu chưa có)

git remote add origin https://github.com/....

git push -u origin main

-u → Thiết lập Mặc định, sau K° cần nữa, nếu muốn thay đổi thì tng tự trên.

main → nhánh

- Git ~~pull~~ fetch: Dùng để tải về các cập nhật mới nhất từ Kho lưu trữ từ xa (Remote) mà không làm thay đổi nhánh hiện tại hoặc thư mục hiện tại của bạn.

1) git fetch origin (remote): Lấy tất cả commit mới từ remote origin.

⇒ Cuối toàn bộ dữ liệu từ remote.

2) git fetch origin main: Cập nhập nhánh orgin/main mà K° làm ảnh hưởng đến main của bạn.

⇒ Chỉ fetch lên 1 nhánh cụ thể.

3) Xem sự khác biệt sau khi fetch: git log main.. origin/main

Hiển thị commit mới trên remote chưa có trong nhánh local.

- Git merge: Hợp nhất các commit từ một nhánh vào nhánh hiện tại.

Ví dụ đang đứng ở nhánh main muốn kết hợp nhánh tung vào thì: git merge tung → Kết nhánh

⇒ Tất cả nội dung của tung chuyển sang main.

Scanned with CamScanner

- Git branch: để quản lý nhánh trong Git.
  - git branch : Hiển thị tất cả các nhánh hiện có trong repo " ....." nhánh đang đứng sẽ có dấu sao "*"
  - Tạo nhánh mới : git branch tên-nhánh
    → Tạo nhánh mới, nhưng bạn vẫn đang ở nhánh cũ
  - git branch -d tên-nhánh : Xoá nhánh nếu đã đc merge
  - git branch -D tên-nhanh : xoá nhanh kể cả chưa merge
  - Đổi tên nhánh: git branh -m tên-cũ tên-mới

Lưu ý: git branch chỉ tạo và quản lý nhánh, k giúp chuyển vị trí nhánh đang đứng.

Muốn chuyển đổi : git checkout tên-nhánh

hoặc cách mới hơn : git switch tên-nhánh

- git checkout: là một lệnh quan (:) trong git ; dùng để chuyển đổi và khôi phục trạng thái
  - Chuyển sang nhánh khác : git checkout tên-nhánh
    Từ Git 2.38 có lệnh : git switch tên-nhánh.
  - Tạo và chuyển sang nhánh mới ngay lập tức : git checkout -b tên-nhá
  - Khôi phục file về trạng thái commit gần nhất. git checkout -- tên_file
  - Chuyển về một commit cụ thể : git checkout <commit-hash>

Khi này repo sẽ ở trạng thái detached HEAD" - tức k đứng tên nhánh nào cả.

- Git pull : Lấy code mới từ remote về và gộp vào nhánh hiện tại.

  git pull = git fetch + git merge.

  1. Git fetch : Tải dữ liệu mới từ ~~ma~~ remote về máy (nhưng chưa gộp)
  2. Git merge: Gộp dữ liệu đó vào nhánh bạn đang đứng

  Cú pháp git pull origin main → tên nhánh mốn lấy về

Scanned with CamScanner